

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Coding Challenge

Course Code:	DSA0404	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5 –Coding Challenge	Assessment:	Assessment Tool 2– Coding Challenge
Weightage:	40% of CIA	Total Marks:	100
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning (BL4)		
Student Name:	Yedururi Anjani Priya	Reg. No.:	192324176
Section / Batch:	2023/AI&DS	Date:	26.08.26

Code of Conduct

I Sania Herbert(192324062) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sania

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

Que 3. Write a function that determines the optimal value of k for a given dataset using k-fold cross-validation, plotting validation error vs. k for k = 1 to 30. Identify and justify the point of the bias-variance trade-off from the curve programmatically (not just visually) — e.g., detect the “elbow” using rate of change.

PYTHON CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import KFold, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
# -----
# 1. Load Dataset
# -----
data = load_breast_cancer()
X = data.data
y = data.target
print("Dataset shape:", X.shape)
# -----
# 2. Define K values
# -----
k_values = range(1, 31)
validation_errors = []
# -----
# 3. K-Fold Cross Validation
# -----
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
```

```

)
for k in k_values:
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    scores = cross_val_score(
        model,
        X,
        y,
        cv=kf,
        scoring="accuracy"
    )
    mean_accuracy = scores.mean()
    error = 1 - mean_accuracy
    validation_errors.append(error)
    print(
        f"k = {k:2d} | "
        f"Accuracy = {mean_accuracy:.4f} | "
        f"Validation Error = {error:.4f}"
    )
# -----
# 4. Find Minimum Validation Error
# -----
minimum_error_index = np.argmin(validation_errors)
best_k_by_error = k_values[minimum_error_index]
print("\nBest k based on minimum validation error:",
      best_k_by_error)
print("Minimum validation error:",
      validation_errors[minimum_error_index])
# -----
# 5. Detect Elbow Programmatically
# Using rate of change
# -----

```

```

errors = np.array(validation_errors)
# First derivative / rate of change
rate_of_change = np.diff(errors)
# Second derivative
second_derivative = np.diff(rate_of_change)
# Elbow is detected where the error curve changes
# its direction/curvature strongly.
elbow_index = np.argmax(np.abs(second_derivative)) + 2
elbow_k = list(k_values)[elbow_index - 1]
print("\nRate-of-change analysis")
print("-----")
for i, rate in enumerate(rate_of_change):
    print(
        f"k={i+1} -> k={i+2} : "
        f"change = {rate:.6f}"
    )
print("\nDetected elbow k:", elbow_k)
# -----
# 6. Plot Validation Error vs k
# -----
plt.figure(figsize=(10, 6))
plt.plot(
    list(k_values),
    validation_errors,
    marker="o"
)
plt.axvline(
    best_k_by_error,
    linestyle="--",
    label=f"Minimum Error k={best_k_by_error}"
)
plt.axvline(
    elbow_k,
    linestyle=":",

```

```

    label=f'Detected Elbow k={elbow_k}'
)
plt.xlabel("Number of Neighbors (k)")
plt.ylabel("Validation Error")
plt.title("kNN: Validation Error vs k")
plt.xticks(range(1, 31))
plt.grid(True)
plt.legend()
plt.show()
# -----
# 7. Final Decision
# -----
print("\nFinal Result")
print("-----")
print("Optimal k based on minimum validation error:",
      best_k_by_error)
print("Programmatically detected elbow:",
      elbow_k)
print("\nBias-Variance Interpretation:")
if elbow_k < best_k_by_error:
    print(
        "The elbow occurs before the minimum-error point. "
        "Increasing k after the elbow provides diminishing "
        "improvements while increasing bias."
    )
elif elbow_k > best_k_by_error:
    print(
        "The minimum validation error occurs before the "
        "detected elbow, indicating that larger k values "
        "begin to increase bias."
    )
else:
    print(
        "The elbow and minimum-error point are approximately "

```

"the same. This is a good balance between bias and variance."
)

OUTPUT:

Dataset shape: (569, 30)

k = 1	Accuracy = 0.9420	Validation Error = 0.0580
k = 2	Accuracy = 0.9455	Validation Error = 0.0545
k = 3	Accuracy = 0.9613	Validation Error = 0.0387
k = 4	Accuracy = 0.9666	Validation Error = 0.0334
k = 5	Accuracy = 0.9595	Validation Error = 0.0405
k = 6	Accuracy = 0.9630	Validation Error = 0.0370
k = 7	Accuracy = 0.9560	Validation Error = 0.0440
k = 8	Accuracy = 0.9578	Validation Error = 0.0422
k = 9	Accuracy = 0.9648	Validation Error = 0.0352
k = 10	Accuracy = 0.9630	Validation Error = 0.0370
k = 11	Accuracy = 0.9595	Validation Error = 0.0405
k = 12	Accuracy = 0.9648	Validation Error = 0.0352
k = 13	Accuracy = 0.9578	Validation Error = 0.0422
k = 14	Accuracy = 0.9578	Validation Error = 0.0422
k = 15	Accuracy = 0.9560	Validation Error = 0.0440
k = 16	Accuracy = 0.9578	Validation Error = 0.0422
k = 17	Accuracy = 0.9543	Validation Error = 0.0457
k = 18	Accuracy = 0.9560	Validation Error = 0.0440
k = 19	Accuracy = 0.9543	Validation Error = 0.0457
k = 20	Accuracy = 0.9543	Validation Error = 0.0457
k = 21	Accuracy = 0.9543	Validation Error = 0.0457
k = 22	Accuracy = 0.9543	Validation Error = 0.0457
k = 23	Accuracy = 0.9543	Validation Error = 0.0457
k = 24	Accuracy = 0.9560	Validation Error = 0.0440
k = 25	Accuracy = 0.9525	Validation Error = 0.0475
k = 26	Accuracy = 0.9490	Validation Error = 0.0510
k = 27	Accuracy = 0.9508	Validation Error = 0.0492
k = 28	Accuracy = 0.9473	Validation Error = 0.0527
k = 29	Accuracy = 0.9525	Validation Error = 0.0475

k = 30 | Accuracy = 0.9508 | Validation Error = 0.0492

Best k based on minimum validation error: 4

Minimum validation error: 0.033410961030895914

Rate-of-change analysis

k=1 -> k=2 : change = -0.003493

k=2 -> k=3 : change = -0.015805

k=3 -> k=4 : change = -0.005279

k=4 -> k=5 : change = 0.007064

k=5 -> k=6 : change = -0.003524

k=6 -> k=7 : change = 0.007033

k=7 -> k=8 : change = -0.001754

k=8 -> k=9 : change = -0.007018

k=9 -> k=10 : change = 0.001754

k=10 -> k=11 : change = 0.003493

k=11 -> k=12 : change = -0.005263

k=12 -> k=13 : change = 0.007018

k=13 -> k=14 : change = 0.000000

k=14 -> k=15 : change = 0.001754

k=15 -> k=16 : change = -0.001754

k=16 -> k=17 : change = 0.003509

k=17 -> k=18 : change = -0.001754

k=18 -> k=19 : change = 0.001754

k=19 -> k=20 : change = 0.000000

k=20 -> k=21 : change = 0.000000

k=21 -> k=22 : change = 0.000000

k=22 -> k=23 : change = 0.000000

k=23 -> k=24 : change = -0.001770

k=24 -> k=25 : change = 0.003524

k=25 -> k=26 : change = 0.003509

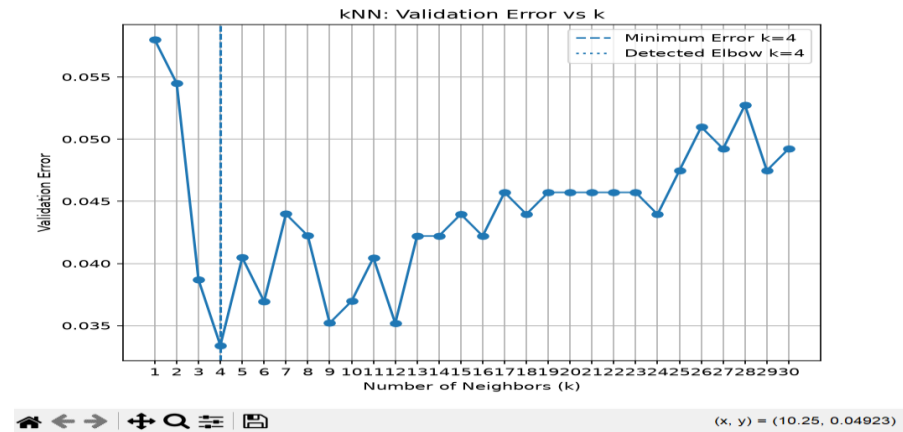
k=26 -> k=27 : change = -0.001754

k=27 -> k=28 : change = 0.003509

k=28 -> k=29 : change = -0.005263

k=29 -> k=30 : change = 0.001754

Detected elbow k: 4



GEOTAG :

